

Department of Electrical and Computer Engineering
COMSATS University Islamabad, Attock Campus

Lab Rubrics Form	
Lab Name	Two-Dimensional Arrays and Matrix Manipulation
Lab Number	12
Submitted by:	
Student Names	Registration #
Sumaira Safeer	FA22-BCE-019

Rubrics		Marks		
		1	2	3
PLO-5 Modern Tools Usage	Ability to execute the experiment utilizing hardware, software tools, or a combination of both.			
PLO-10 Communication Skills	Present lab activities effectively through detailed lab reports, oral examinations, and engaging presentations.			

Lab 12: Two-Dimensional Arrays and Matrix Manipulation

Objective

To understand and implement matrix operations using two-dimensional arrays and nested loops in C programming, including matrix addition and multiplication.

Software Requirements

- Code::Blocks IDE / Dev-C++
- GCC Compiler
- Windows Operating System

Experiments Performed

In-Lab Tasks

1. Add two 3×3 matrices using two-dimensional arrays.
2. Multiply two 3×3 matrices entered by the user and display the resultant matrix.

Post-Lab Task

1. Develop a generic matrix multiplication program that can multiply matrices of any valid dimensions by taking the matrix sizes and elements from the user.

Learning Outcomes

After completing this lab, the student will be able to:

- Understand the concept of two-dimensional arrays.
- Perform matrix addition and multiplication.
- Use nested loops for multidimensional data processing.
- Implement matrix algorithms efficiently.
- Develop generalized programs for matrix operations.
- Apply array manipulation techniques to solve mathematical problems.

In-Lab Tasks:

Task1:

Write a program to add two 3*3 matrices.

Code:

```
#include <stdlib.h>

void addMatrices(int mat1[][3], int mat2[][3], int result[][3])
{
    int i, j;
    for (i=0; i<3; i++) {
        for (j= 0; j<3; j++) {
            result[i][j] = mat1[i][j] + mat2[i][j];
        }
    }
}

void displayMatrix(int matrix[][3])
{
    int i,j;
    for (i=0; i<3; i++) {
        for (j=0; j<3; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

int main()
{
    int mat1[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    int mat2[3][3] =
    {
```

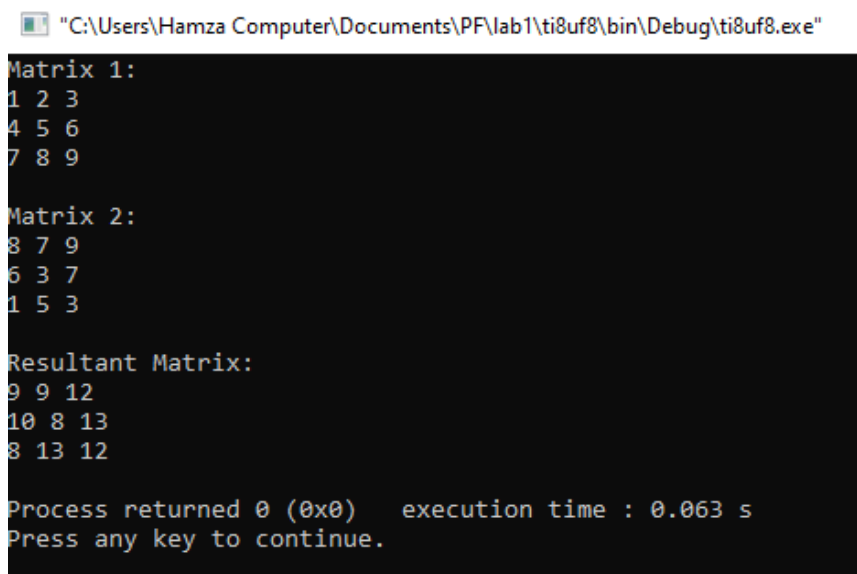
```

        {8, 7, 9},
        {6, 3, 7},
        {1, 5, 3}
    };

    int result[3][3];
    addMatrices(mat1, mat2, result);
    printf("Matrix 1:\n");
    displayMatrix(mat1);
    printf("\nMatrix 2:\n");
    displayMatrix(mat2);
    printf("\nResultant Matrix:\n");
    displayMatrix(result);
    return 0;
}

```

Output:



```

"C:\Users\Hamza Computer\Documents\PF\lab1\ti8uf8\bin\Debug\ti8uf8.exe"
Matrix 1:
1 2 3
4 5 6
7 8 9

Matrix 2:
8 7 9
6 3 7
1 5 3

Resultant Matrix:
9 9 12
10 8 13
8 13 12

Process returned 0 (0x0)   execution time : 0.063 s
Press any key to continue.

```

Task2:

Write a program to multiply any two 3x3 matrices. Take these arrays from the user. Make the flow chart for this task. Do the dry run for one iteration of outer loop.

Code:

```
#include <stdlib.h>

void multiplyMatrices(int mat1[][3], int mat2[][3], int result[][3])
{
    int i, j, k;
    for(i=0; i<3; i++) {
        for (j=0; j<3; j++) {
            result[i][j] = 0;
            for (k=0; k<3; k++)
            {
                result[i][j] += mat1[i][k] * mat2[k][j];
            }
        }
    }
}

void displayMatrix(int matrix[][3])
{
    int i,j;
    for (i=0; i<3; i++) {
        for (j= 0; j<3; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

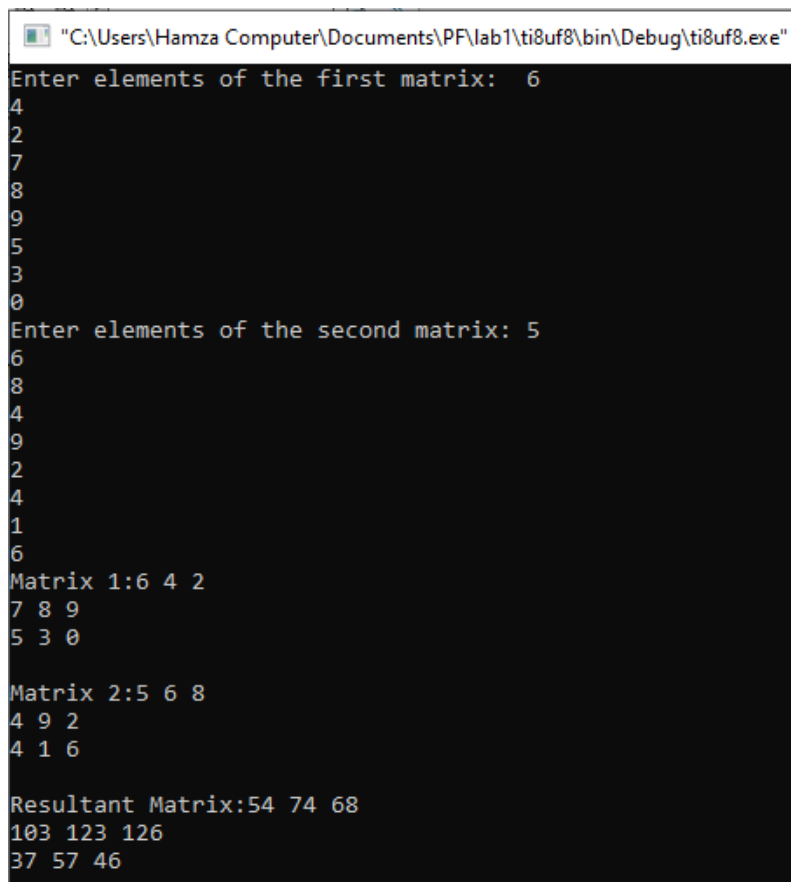
int main()
{
    int mat1[3][3];
    int mat2[3][3];
    int result[3][3];
    printf("Enter elements of the first matrix:");
    int i, j;
    for (i=0; i<3; i++) {
        for (j=0; j<3; j++) {
            scanf("%d", &mat1[i][j]);
        }
    }
}
```

```

    }}
    printf("Enter elements of the second matrix:");
    for (i=0; i<3; i++) {
        for (j=0; j<3; j++) {
            scanf("%d", &mat2[i][j]);
        }
    }
    multiplyMatrices(mat1, mat2, result);
    printf("Matrix 1:");
    displayMatrix(mat1);
    printf("\nMatrix 2:");
    displayMatrix(mat2);
    printf("\nResultant Matrix:");
    displayMatrix(result);

```

Output:



```

"C:\Users\Hamza Computer\Documents\PF\lab1\ti8uf8\bin\Debug\ti8uf8.exe"
Enter elements of the first matrix: 6
4
2
7
8
9
5
3
0
Enter elements of the second matrix: 5
6
8
4
9
2
4
1
6
Matrix 1:6 4 2
7 8 9
5 3 0
Matrix 2:5 6 8
4 9 2
4 1 6
Resultant Matrix:54 74 68
103 123 126
37 57 46

```

Post-Lab Tasks:

Task1:

Write a generic code for multiplying 2 matrices by considering the following terms.

Code:

```
def multiply_matrices(matrix1, matrix2):
    rows1 = len(matrix1)
    cols1 = len(matrix1[0])
    rows2 = len(matrix2)
    cols2 = len(matrix2[0])
    if (cols1 != rows2)
print("Error: The number of columns in 1st = to number of rows in 2nd matrix.");
    result = [[0] * cols2 for _ in range(rows1)];

    for i in range(rows1);
        for j in range(cols2);
            for k in range(cols1);
                result[i][j] += matrix1[i][k] * matrix2[k][j];
    return result;

rows1 = int(input("Enter number of rows for 1stmatrix: "));
cols1 = int(input("Enter number of columns for 1st matrix: "));
rows2 = int(input("Enter number of rows for 2nd matrix: "));
cols2 = int(input("Enter number of columns for 2ndmatrix: "));
print("Enter the elements of 1st matrix:");
matrix1 = [[int(input()) for _ in range(cols1)] for _ in range(rows1)];
print("Enter the elements of 2nd matrix:");
matrix2 = [[int(input()) for _ in range(cols2)] for _ in range(rows2)];
result = multiply_matrices(matrix1, matrix2);
```

```
print("Matrix multiplication result:");  
    print(row);
```

Output:

```
Enter the number of rows for the first matrix: 2  
Enter the number of columns for the first matrix: 3  
Enter the number of rows for the second matrix: 3  
Enter the number of columns for the second matrix: 2
```

```
Enter the elements of the first matrix:
```

```
1  
2  
3  
4  
5  
6
```

```
Enter the elements of the second matrix:
```

```
7  
8  
9  
10  
11  
12
```

```
Matrix multiplication result:
```

```
[58, 64]  
[139, 154]
```